Что уже сделано и первые замеры

Студент	Евстигнеев Н. А., группа P3431
Тема	«Разработка ИИ-агента для приложений граничного искусственного интеллекта»
Дата	7 октября 2026 г.

> **Важная оговорка.** Все числа ниже — **предварительные ориентиры**. Прогоны шли на модели-заглушке (без реального API), в виртуальных машинах и при параллельной нагрузке на ноутбук. Они показывают, что стенд работает и в какую сторону идут эффекты. Итоговые результаты будут получены по протоколу: на свободной машине, с повторами, на реальной модели и на плате.

1. Программная часть

Компонент	Назначение	Язык	Код / тесты, строк
Агент EdgeCtx	часы (реальные и виртуальные), событийный фильтр и правила, планировщик с бюджетом, контексты пользователя и оборудования, сборка запроса, клиент модели, MCP-клиент, журнал трасс	Go	7 927 / 3 643
MCP-сервер оборудования	датчики и GPIO из данных ядра Linux, два варианта каталога инструментов (общие инструменты или отдельный инструмент на каждое свойство), проверка команд приводам, аренда управления с возвратом в безопасное состояние	Go	7 293 / 3 540
MCP-адаптер робота	статус, асинхронная поездка, обход, просмотр (сводка или кадр по запросу), остановка; события вместо опроса	Go	5 501 / 1 625
Монитор ресурсов (sysmon)	память (PSS/RSS), процессор, ввод-вывод, трафик процессов и cgroup во времени	Go	2 835 / 1 271
Стенд «рабочего дня»	генератор дня (CASAS + частоты команд), эмулятор помещения, модель-заглушка, проигрыватель дня, метрики, модель стоимости, статистика, монитор трасс, обвязка аналогов	Python	≈17 000 / ≈8 100
Стенд робота	создание и настройка виртуальной машины, запуск симулятора, замеры нагрузки и трафика	shell, Python	≈3 700 / —

Всего около 62 тыс. строк кода и тестов; сейчас все тесты проходят (Go — 46 пакетов, Python — 394 теста). **Взято готовым**, а не разработано мной: ROS 2, Gazebo, TurtleBot3, Nav2 и rosbridge (симулятор робота); официальные MCP SDK для Go и Python; аналоги PicoClaw, nanobot и ros-mcp-server; записи датчиков CASAS и фразы команд MASSIVE; tiktoken для подсчёта токенов. Серверы оборудования и робота, стенд «рабочего дня», сэмплер, монитор и обвязка аналогов прошли независимое ревью с замерами; ревью

самого агента было прервано и будет повторено. Часть исправлений уже внесена, но ещё не перепроверена; найденные ошибки — в списке ниже.

2. Стенд

- **Виртуальная плата**: Ubuntu 24.04, ARM, 4 ядра, около 4 ГБ памяти. Проверено, что ограничения работают: при лимите 32 МиБ процесс, которому нужно 64 МиБ, завершается системой, как на настоящей плате; квота 10 % ядра даёт ровно 0,3 с процессорного времени за 3 с; доступны эмуляция плохой сети и медленного диска.
- **Симулятор робота**: отдельная виртуальная машина с ROS 2 Jazzy, Gazebo Harmonic, TurtleBot3 (камера, лидар, одометрия) и навигацией Nav2. С «платы» робот управляется по сети через MCP-адаптер (пока тестовым MCP-клиентом; подключение агента — следующий шаг). В ускоренном режиме (примерно в 2,5 раза быстрее реального времени) робот под управлением Nav2 дошёл до всех 15 целей из 15.
- **Аналоги**: PicoClaw 0.3.1 и nanobot 0.3.5 установлены (версии и контрольные суммы зафиксированы) и подключены к той же модели-заглушке и тем же MCP-серверам; реплики и события доходят до них через их штатные каналы.
- **Данные**: записи датчиков реальных домов CASAS (открытый набор, лицензия CC BY 4.0).

3. Потребление ресурсов компонентами

Компонент	Память	Прочее
Агент EdgeCtx	≈13 МиБ в простое (виртуальная плата, PSS); ≈26–29 МиБ при проигрывании рабочего дня (ноутбук, RSS; на плате ещё не мерилось)	исполняемый файл ≈13,5 МБ; единичные пробуждения в минуту в простое
MCP-сервер оборудования	≈9–10 МиБ в простое, ≈15 МиБ под нагрузкой, без утечек	проверка команды приводу ≈0,5 мкс
MCP-адаптер робота	≈11–15 МиБ	запуск ≈3 мс; исполняемый файл ≈9,5 МБ
Монитор ресурсов	≈9 МиБ	≈0,2 % ядра при опросе раз в секунду; ≈0,007 % при опросе раз в 30 с

4. Первое сравнение с аналогами (пробный прогон)

Двухчасовой фрагмент рабочего дня, сжатый в 60 раз, модель-заглушка с «правильными ответами», одинаковый сценарий и эмулятор помещения; у аналогов в части прогонов подключён ещё и сервер оборудования (у разработанного агента — нет), на равной основе сравнивают прогоны «только эмулятор».

Система	Вызовов модели	Входных токенов	Входных токенов на вызов (при ответе на реплику)	Память

Разработанный агент	77	≈0,20 млн	≈2,7 тыс.	≈28 МиБ (без сервера оборудования)
PicoClaw, профиль «best» (seahorse)	91	≈0,62–0,64 млн	≈8,1–8,3 тыс.	≈42 МиБ без сервера оборудования; ≈48–59 МиБ с ним
PicoClaw без seahorse и с выключенными лишними инструментами (перепрогон ревьюера)	102	≈0,39 млн	—	—
nanobot	90	≈0,84–0,90 млн	≈10,8–11,5 тыс.	≈166 МиБ без сервера оборудования; ≈167–188 МиБ с ним

Что показало ревью этого сравнения (честно). Настройка PicoClaw, которую мы считали лучшей, по экономности такой не была: без seahorse и с выключенными лишними встроенными инструментами (это штатные настройки PicoClaw) преимущество разработанного агента по сумме входных токенов — примерно в 1,9 раза, а не в 3,1; у PicoClaw при этом 102 вызова против 77. Это временный перепрогон ревьюера, на стенде его ещё надо повторить. У nanobot сжатие времени в 60 раз выключило фоновое сжатие истории; при сжатии в 15 раз у него 96 вызовов и ≈0,78 млн входных токенов. В первой версии сравнения память и токены сравнивались на разной основе (у аналогов — вместе с сервером оборудования); в таблице выше основа уже одинаковая. Настройка PicoClaw и режим сжатия исправляются в обвязке аналогов до основных прогонов.

5. Робот: свой адаптер против готового ros-mcp-server

Показатель	Свой адаптер	ros-mcp-server
Описание инструментов в каждом запросе	≈274 токена (5 инструментов)	≈2 221 токен (31 инструмент)
Ответ на «где ты?» / «что впереди?»	≈52 токена (одна сводка на оба вопроса)	≈235 токенов (сырая поза) / ≈4 433 токена (сырой скан лидара)
Входные токены на задачу	в 3,7–10 раз меньше у своего адаптера (например, «что впереди?»: 906 против 9 201)	
Трафик на одну поездку (≈3,7 м)	≈4–8 КБ	≈926 КБ
Трафик в простое (2 мин)	0	—
Кадр с камеры по запросу	10 из 10, ≈0,7 с	4 из 10 (новое соединение на каждый вызов: с «ленивой» камерой стенда первый кадр не успевает прийти)

Команда «стоп» (при живой связи)	≈0,5 мс, робот проезжает по инерции 2–4 см	≈2 с, ≈15 см
Память процесса на плате	≈11–15 МиБ	≈110 МиБ
Время запуска	≈3 мс	≈431 мс

В плане ВКР (гипотеза H6) стоит более ранний ориентир — по прототипу адаптера на Python: в 5,6–11 раз меньше входных токенов и в 50–70 раз меньше трафика. Здесь — итоговая версия на Go: каталог у неё длиннее (274 токена против 229), а асинхронная поездка стоит +1 вызов модели, поэтому выигрыш по токенам немного меньше (3,7–10 раз), а по трафику больше (≈110–220 раз). Ориентир в плане будет обновлён.

Главный вывод: сырые данные робота нельзя отдавать модели — одно сообщение лидара стоит около 4,4 тыс. токенов, две секунды подписки — около 109 тыс. Короткие сводки и события сокращают и токены, и трафик на порядки.

6. Найденные ревью ошибки и что с ними

- **Стенд «рабочего дня»**: несовпадение формата ресурсов эмулятора помещения и сервера оборудования, неучтённые служебные заголовки и запросы сводки в заглушке модели, единицы в одной из метрик. Исправлено, добавлен сквозной тест «настоящий агент + эмулятор + заглушка»; исправления ещё не прошли повторное ревью, поэтому полные дни — после него.
- **Сравнение с аналогами**: в цифрах раздела 4 уже учтены самая экономная настройка PicoClaw, меньшее сжатие времени для nanobot и единая основа сравнения памяти; в обвязке аналогов это ещё нужно сделать настройками по умолчанию.
- **Сэмплер и монитор**: в режиме по умолчанию не записывается завершение процесса из-за нехватки памяти (нужно для «лестницы ограничений»); проверка безопасности команд в мониторе даёт ложные срабатывания на настоящем журнале сервера оборудования.
- **Сервер оборудования**: уведомления тормозили опрос датчиков и сторожевой таймер, а таймеры давали лишние пробуждения в простое (тик 100 мс — в 5 раз больше процессорного времени и в 6 раз больше пробуждений). **Исправлено**: уведомления рассылаются асинхронно, таймеры спят до ближайшего срока (в простое без аренд — ноль пробуждений таймеров), при остановке приводы сразу уходят в безопасное состояние; добавлен регрессионный тест на «зависшего» клиента. Осталось: поддержка старого интерфейса GPIO ядра 4.9, нужного для Jetson Nano.
- **Адаптер робота**: сейчас при обрыве связи или остановке адаптера цель навигации не отменяется — робот доезжает до неё сам, а «стоп» отвечает «ok»; если робота сдвинуть не через адаптер, он сообщает устаревшее место и старый скан. Нужно отменять цель (проверенное исправление — отмена всех целей сервисом Nav2, 4,9 мс); это обязательно до любой работы с реальным роботом.

7. Что дальше

1. До конца октября — дымовой тест кафедрального Jetson Nano (запуск агента, PicoClaw, nanobot и детектора, доступ к RouterAI из сети ИТМО) и решение о плате.
2. Исправить найденные ошибки стенда и обеспечить равные условия сравнения; провести повторное ревью.
3. Первые полные ускоренные дни: периодическая проверка и событийный фильтр, разные политики контекста; сравнение с аналогами.
4. Рабочий день робота; сервер камеры с детектором на плате.
5. Лестница ограничений целиком («карта выживаемости»).
6. С ключом RouterAI: проверка на реальной модели (качество ответов, фактический расход токенов и работа кэша).
7. С доступом к Jetson Nano: перенос и реальные 8-часовые прогоны, калибровка виртуальной платы.
8. Параллельно — текст пояснительной записки: глава 1 (обзор и постановка задачи) к 15 января.